

Network And Deployment Architecture

Date updated: 2026-06-03

Purpose

This document explains how clients reach the backend, where TLS is applied, how the gateway/Nginx/FastAPI/PostgreSQL layers connect, which ports are exposed, and which network controls support the Computer Networks and Cybersecurity part of the CS4455 brief.

It is paired with:

- `docs/security/network_deployment_threat_model.md` for network threats and residual risks.
- `docs/deployment/runbook.md` for operational commands and evidence collection.
- `docs/security/security_test_results.md` for final test and scan results.

Marking-Focused Summary

Brief/rubric item	Network/deployment evidence
Secure client/server connectivity	Public clients use <code>https://kfc.theburkenator.com</code> ; C++ real mode requires HTTPS and keeps Qt certificate validation active.
Certificate authenticity and validity	Client validates hostname/certificate; backend TLS probe verifies DNS, SNI hostname, certificate subject/issuer/expiry, TLS version, and cipher.
Network architecture documentation	This document records DNS, TLS gateway, internal HTTP boundary, Nginx reverse proxy, Uvicorn local binding, PostgreSQL local binding, ports, headers, rate limits, and external services.
Backend processing	Nginx proxies API requests to local FastAPI; FastAPI authenticates, validates, authorizes, rate limits, and uses PostgreSQL.
Server-side security controls	Nginx edge rate limits, FastAPI HTTPS/HSTS semantics, security headers, CORS hardening, localhost-only services, and systemd environment handling.
Vulnerability testing evidence	Security header tests, rate-limit tests, TLS probe utility, exposed-port runbook checks, Bandit, pip-audit, and DB-backed pytest evidence.

Public Entry Point

External users access the backend through:

`https://kfc.theburkenator.com`

API version prefix:

`/api/v1`

OpenAPI/Swagger documentation:

`https://kfc.theburkenator.com/docs`

The C++ client default API URL is:

`https://kfc.theburkenator.com/api/v1`

End-to-End Deployment Shape



Public internet traffic is HTTPS to the gateway. Internal VM services bind locally behind Nginx.

End-To-End Deployment Shape

Key points:

- The public internet path is protected by HTTPS up to the provided TLS gateway.
- TLS terminates at the gateway, not inside FastAPI.
- The gateway-to-VM hop is internal HTTP to VM port 80.
- Nginx is the VM process that receives gateway-forwarded traffic.
- Uvicorn/FastAPI listens on `127.0.0.1:8000`, not a public interface.
- PostgreSQL listens on `127.0.0.1:5432`, not a public interface.
- The blockchain worker is separate from request handling and is the only backend component that calls Sepolia.

Components

Component	Role	Binding / endpoint	Security behaviour
DNS/subdomain	Routes users to team service	kfc.theburkenator.com	Users and clients use the assigned hostname, not an IP address or lookalike domain.
C++ client	Real secure messaging client	https://kfc.theburkenator.com/api/v1	Real mode rejects non-HTTPS URLs and keeps Qt certificate validation active.
Browser/API client	Manual/API testing	https://kfc.theburkenator.com	Platform/browser verifies the gateway certificate.
Provided TLS gateway	Public HTTPS termination	Public 443	Presents public certificate and forwards to VM over internal HTTP.
Nginx	VM reverse proxy and edge limiter	VM :80	Applies limit_req, sets proxy headers, forwards to local Uvicorn.
Uvicorn/FastAPI	Backend API	127.0.0.1:8000	Authenticates users, validates inputs, authorizes objects, emits headers.

Component	Role	Binding / endpoint	Security behaviour
PostgreSQL Docker	Database	127.0.0.1:5432	Stores accounts, token-session hashes, public keys, ciphertext, anchors, audit logs.
Blockchain worker	Async anchor submitter	local DB plus outbound Sepolia RPC	Reads pending anchors, submits contract transaction, records confirmation metadata.
SSH	Operator access	22/tcp, restricted	Used for deployment, logs, service checks, and evidence commands.

Request Flow

1. The client uses the assigned hostname `kfc.theburkenator.com`.
2. The client establishes HTTPS to the public gateway.
3. The client validates the certificate chain and hostname.
4. The gateway terminates TLS and forwards the request to the team VM on port 80.
5. Nginx receives HTTP on port 80.
6. Nginx applies route-specific `limit_req` controls.
7. Nginx sets proxy headers and forwards to `http://127.0.0.1:8000`.
8. FastAPI middleware checks HTTPS semantics when configured, adds security headers, and sanitizes validation errors.
9. FastAPI route dependencies validate Bearer tokens and apply authenticated per-user rate limits where required.
10. Pydantic schemas validate request bodies, path parameters, and query parameters.
11. Services perform business rules and object-level authorization.
12. Repositories query PostgreSQL through SQLAlchemy/asyncpg.

13. Response schemas serialize only allowed fields.

Trust Boundaries

Boundary	What crosses it	Control
Client to DNS/ hostname	Hostname lookup and selected endpoint	The documented/ default endpoint is <code>kfc.theburkenator.com</code> ; certificate hostname validation catches wrong-host certificates.
Client to TLS gateway	Credentials, tokens, public keys, ciphertext, anchor metadata	HTTPS with certificate validation.
Gateway to VM	HTTP request after TLS termination	Internal infrastructure boundary; Nginx is the VM listener; backend docs state this residual boundary clearly.
Nginx to FastAPI	HTTP loopback proxy traffic	Uvicorn binds to <code>127.0.0.1</code> ; Nginx sets trusted proxy headers.
FastAPI to PostgreSQL	DB queries and application data	PostgreSQL bound to localhost; credentials in <code>.env</code> ; SQLAlchemy ORM.
Worker to Sepolia	Signed transaction and RPC calls	Worker process owns wallet env values; FastAPI routes do not hold wallet credentials.

DNS And Certificate Verification

The service relies on the assigned subdomain:

`kfc.theburkenator.com`

Client-side controls:

- The C++ client default API URL is `https://kfc.theburkenator.com/api/v1`.
- `StartupConfig.cpp` rejects non-HTTPS API URLs in real mode.
- `HttpGateways.cpp` uses Qt networking and does not call `ignoreSslErrors()`, so Qt certificate validation remains active.
- TLS handshake errors map to the client's TLS error path.
- Browser/API clients use normal platform certificate validation.

Operator evidence:

```
cd server/backend
.venv/bin/python scripts/check_tls_connection.py kfc.theburkenator.com
--port 443
```

The TLS probe records:

- resolved peer address
- verified SNI/hostname
- TLS version
- cipher suite
- certificate subject
- certificate issuer
- certificate expiry

TLS Gateway

The provided gateway is the public TLS endpoint. It accepts HTTPS on 443 for `kfc.theburkenator.com`, presents the public certificate, and forwards traffic to the VM on port 80.

Security consequences:

- Public internet traffic is confidential and integrity-protected up to the gateway.
- FastAPI does not directly manage the public certificate.
- Certificate renewal and gateway TLS policy are operational responsibilities of the gateway environment.
- Backend evidence uses the TLS probe and public `curl -I` checks to prove the deployed certificate path.

FastAPI defense-in-depth:

- `ENFORCE_HTTPS=true` lets FastAPI reject plain HTTP requests at the application layer.
- `TRUST_X_FORWARDED_PROTO=true` lets the trusted gateway/Nginx path signal that the original public request was HTTPS.
- `HSTS_ENABLED=true` emits Strict-Transport-Security on HTTPS responses.

Internal HTTP Gateway-To-VM Boundary

The gateway-to-VM hop is internal HTTP because TLS is terminated at the provided gateway.

This is an explicit deployment boundary:

- The public internet side remains HTTPS.
- The VM receives gateway-forwarded HTTP on port 80.
- Nginx is the VM process that handles this traffic.
- Uvicorn and PostgreSQL are not public listeners.
- Application-layer protections still apply after the request reaches FastAPI: JWT signatures, validation, authorization, rate limits, and end-to-end encrypted message payloads.

Residual risk:

- Confidentiality and integrity on the gateway-to-VM hop depend on the provided infrastructure. This is documented in the threat model and residual risk register.

Nginx To FastAPI Boundary

Nginx proxies to local FastAPI:

```
upstream epic_messaging_api {
    server 127.0.0.1:8000;
    keepalive 16;
}
```

Important proxy settings from `backend/deploy/nginx/epic-messaging-api.conf`:

```
server {
    listen 80;
    server_name kfc.theburkenator.com;

    client_max_body_size 1m;
```

```

    limit_req_status 429;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
}

```

Security behaviour:

- Nginx is the only VM HTTP listener for gateway traffic.
- Uvicorn runs on localhost only through `epic-messaging-api.service`.
- Nginx sets proxy headers rather than trusting arbitrary client-provided values.
- Real-IP handling is configured only for trusted gateway IPs when the gateway provides real client IPs.
- FastAPI trusts forwarded scheme headers only when configured for the trusted gateway deployment.

VM Ports

Expected deployment exposure:

Port	Binding/ exposure	Purpose	Evidence command
80/tcp	VM listener for gateway/internal traffic	Nginx reverse proxy	<code>sudo ss -tulpn grep ' :80 '</code>
8000/tcp	127.0.0.1 only	Uvicorn/FastAPI	<code>sudo ss -tulpn grep ' :8000 '</code>
5432/tcp	127.0.0.1 only	PostgreSQL Docker	<code>sudo ss -tulpn grep ' :5432 '</code>
22/tcp	restricted administration	SSH	VM/firewall/SSH config

Uvicorn service evidence:

```
ExecStart=.../uvicorn app.main:app --host 127.0.0.1 --port 8000
```

PostgreSQL binding evidence:

```
docker run --name epic-postgres \
  -e POSTGRES_DB=secure_messages \
  -e POSTGRES_USER=secure_app_user \
  -e POSTGRES_PASSWORD=<database-password> \
  -p 127.0.0.1:5432:5432 \
  -d postgres:16
```

Layered Rate Limiting

The deployment uses Nginx before FastAPI so obvious floods are rejected before Python application work is spent.

Nginx edge limits:

Route group	Nginx zone	Rate
Register	epic_register_ip	1r/m
Login	epic_login_ip	1r/m
Refresh	epic_refresh_ip	2r/m
Prekey bundle fetch	epic_prekey_ip	60r/m
Message send/forward	epic_message_ip	120r/m
General API/docs	epic_api_ip	300r/m

FastAPI limits:

Layer	Identity	Purpose
Unauthenticated route limits	direct client IP from request	Register/login/refresh abuse
Authenticated route limits	current_user.id	Per-user message/key/blockchain abuse

Gateway note:

- If the provided gateway forwards all users from one source IP, Nginx sees that gateway address.
- Real-IP handling is restricted to trusted gateway IPs only.

- Arbitrary public X-Forwarded-For headers are not trusted.

CORS, HTTPS, And Security Headers

FastAPI app-level controls:

Setting/control	Behaviour
ALLOWED_ORIGINS	CORS middleware is installed only when configured.
APP_ENV=production	Rejects wildcard origins.
CORS_ALLOW_CREDENTIALS=true with wildcard	Rejected by settings validation.
ENFORCE_HTTPS=true	Returns 426 HTTPS is required for non-HTTPS requests.
TRUST_X_FORWARDED_PROTO=true	Accepts trusted gateway/Nginx HTTPS signalling.
HSTS_ENABLED=true	Emits Strict-Transport-Security on HTTPS responses.
SECURITY_HEADERS_ENABLED=true	Adds no-sniff, frame-deny, no-referrer, no-store, and permissions-policy headers.

Security headers emitted by middleware:

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- Referrer-Policy: no-referrer
- Cache-Control: no-store
- Permissions-Policy: geolocation=(), microphone=(), camera=()
- Strict-Transport-Security on HTTPS responses when HSTS is enabled

PostgreSQL Connectivity

The backend uses:

```
postgresql+asyncpg://...
```

Security assumptions:

- PostgreSQL is reachable only from the backend host.
- Database credentials are loaded from `.env`.
- Real `.env` values are not committed.
- Alembic migrations are applied before starting/restarting the API.
- `TEST_DATABASE_URL` points to a separate test database whose name contains `test`.

Operational checks:

```
docker ps
docker logs epic-postgres --tail 100
alembic current
sudo ss -tulpn | grep ':5432'
```

Blockchain Worker Connectivity

FastAPI message routes create pending `blockchain_anchors` rows in PostgreSQL. The separate worker submits those rows to Sepolia.

Worker network path:

```
blockchain worker -> PostgreSQL localhost -> Sepolia RPC over HTTPS ->
MessageFidelity contract
```

Worker environment values:

```
BLOCKCHAIN_WORKER_ENABLED=true
SEPOLIA_RPC_URL=<sepolia-rpc-url>
DEPLOYER_PRIVATE_KEY=<sepolia-worker-private-key>
MESSAGE_FIDELITY_ADDRESS=<message-fidelity-contract-address>
```

Security behaviour:

- FastAPI request handlers do not hold wallet credentials.
- FastAPI request handlers do not block on Sepolia confirmation.
- Pending anchors remain in PostgreSQL during worker/RPC/Sepolia outages.
- The worker updates `status`, `transaction_hash`, `contract_address`, and `anchored_at` after confirmation.

External Services

The backend deployment depends on:

Service	Used by	Purpose
---------	---------	---------

Service	Used by	Purpose
PostgreSQL	FastAPI and worker	Application persistence.
Sepolia RPC	Blockchain worker	Contract submission for pending anchors.

FastAPI request routes do not call an email provider, cloud KMS, third-party messaging service, or live Sepolia RPC.

Evidence Checklist

Evidence	Command/file
Public TLS certificate	<code>.venv/bin/python scripts/ check_tls_connection.py kfc.theburkenator.com --port 443</code>
C++ client HTTPS-only real mode	<code>client/src/app/ StartupConfig.cpp, client/src/ support/ClientConstants.h, client/README.md</code>
Nginx active config	<code>sudo nginx -T grep -n "server_name\\ proxy_pass\\ listen\\ limit_req"</code>
Nginx syntax	<code>sudo nginx -t</code>
API service	<code>sudo systemctl status epic- messaging-api</code>
Worker service	<code>sudo systemctl status epic- messaging-blockchain-worker</code>
Public listener	<code>sudo ss -tulpn grep ':80'</code>
Local Uvicorn binding	<code>sudo ss -tulpn grep ' :8000 '</code>
Local PostgreSQL binding	<code>sudo ss -tulpn grep ' :5432 '</code>
HTTPS headers	<code>curl -I https:// kfc.theburkenator.com/docs</code>
Nginx rate limit smoke check	repeated login curl in docs/ deployment/runbook.md returns 401 then 429
Final tests/scans	<code>docs/security/ security_test_results.md</code>

Residual Network Risks

Residual risk	Control/boundary
Public TLS terminates at the provided gateway	Public internet traffic remains HTTPS; the internal hop is documented.
Gateway-to-VM traffic is internal HTTP	Nginx is the VM listener; Uvicorn/ PostgreSQL remain local; application-layer security still applies.
Gateway can hide real client IP	Nginx real-IP handling is restricted to trusted gateway addresses; FastAPI has authenticated per-user limits.
FastAPI limiter is in-memory	Nginx provides edge limits for the single-VM deployment.
Certificate renewal and gateway configuration	TLS probe and public header checks capture deployed evidence; renewal is operational.
PostgreSQL and FastAPI share one VM	Local binding reduces network exposure; VM compromise remains high impact.
Worker/RPC/Sepolia outage	Pending anchors persist in PostgreSQL and are processed when the worker resumes.